



ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA ĐIỆN TỬ – VIỄN THÔNG

BÁO CÁO ĐỒ ÁN
HỌC PHẦN CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

HeapBeat
Phát Nhạc Chờ Cộng Đồng
Cho Phòng Tự Học
(COLLABORATIVE DJ QUEUE)

Hệ thống xếp hàng ưu tiên và bình chọn bài hát

Nhóm sinh viên thực hiện:

HeapBeat

24207043	Phạm Hùng Tiến
24207029	Trần Thái Nguyên
24207105	Quách Gia Thịnh
24207110	Đặng Gia Toàn
24207002	Phạm Cao Bằng
24207111	Lê Quang Bảo Trung
24207090	Văn Phú Duy
24207101	Hồ Sĩ Phú

Thành phố Hồ Chí Minh, ngày 08 tháng 08 năm 2026

THÔNG TIN BÁO CÁO

Hạng mục	Thông tin
Học phần	Cấu trúc dữ liệu và Giải thuật
Loại tài liệu	Báo cáo đồ án khoa học kỹ thuật
Đề tài	HeapBeat – Phát nhạc chờ cộng đồng cho phòng tự học (Collaborative DJ Queue)
Trạng thái	Bản nộp học phần, tháng 8 năm 2026
Ngày thuyết trình	08/08/2026
Kho mã nguồn	github.com/PhamHungTien/HeapBeat
Giấy phép mã nguồn	MIT License
Nền tảng	React, TypeScript, Web/PWA; mô hình đối chiếu bằng C11
CTDL trọng tâm	Max-Heap + Index Map; danh sách liên kết đôi vòng; Hash Map
Kết quả kiểm tra	22/22 kiểm thử đơn vị đạt; bản C chạy đúng; bản dựng web hoàn tất

Danh sách nhóm và gói công việc chính

MSSV	Thành viên	Gói công việc chính
24207043	Phạm Hùng Tiến	Kiến trúc trạng thái, tích hợp và đóng gói
24207029	Trần Thái Nguyên	CDLL và điều khiển Next/Prev/Repeat
24207105	Quách Gia Thịnh	Max-Heap, bộ so sánh, bình chọn và xóa
24207110	Đặng Gia Toàn	SpamGuard, cửa sổ trượt, chặn và thu hồi
24207002	Phạm Cao Bằng	Giao diện thích ứng và khả năng truy cập
24207111	Lê Quang Bảo Trung	Cổng API PHP-C và snapshot JSON
24207090	Văn Phú Duy	Kế hoạch kiểm thử, bản C và tổng hợp số liệu
24207101	Hồ Sĩ Phú	Danh mục nhạc, giấy phép và báo cáo

TÓM TẮT

Đề tài xuất phát từ nhu cầu tổ chức một danh sách phát chung trong phòng tự học, nơi nhiều sinh viên có thể đề xuất và bình chọn bài hát nhưng không được làm gián đoạn trải nghiệm của những người còn lại. Bài toán đặt ra ba yêu cầu chính: xác định nhanh bài có mức ưu tiên cao nhất, duyệt lịch sử phát theo hai chiều và hạn chế các yêu cầu lặp lại từ cùng một tài khoản. Trên cơ sở đó, nhóm xây dựng **HeapBeat**, một ứng dụng web/PWA sử dụng Max-Heap kèm bản đồ chỉ số cho hàng đợi bình chọn, danh sách liên kết đôi vòng cho lịch sử phát và Hash Map theo mã sinh viên cho cơ chế chống gửi yêu cầu quá mức.

Hệ thống dùng React 19 và TypeScript cho giao diện, còn backend C11 là nguồn sự thật thực thi ba cấu trúc dữ liệu. Độ đúng đắn được xem xét thông qua bất biến, độ phức tạp tiệm cận, 22 kiểm thử đơn vị, 56 assertion backend C và các tình huống sử dụng trên trình duyệt. Kết quả cho thấy cả 22 kiểm thử đều đạt; thao tác xem phần tử đầu Heap, Next/Prev và tra cứu trạng thái chống spam có chi phí $O(1)$ (trung bình đối với Hash Map), trong khi thêm, lấy, xóa và cập nhật bình chọn có chi phí $O(\log n)$. Ở các kích thước màn hình đã kiểm tra, bảng hàng đợi không phát sinh cuộn ngang. Báo cáo đồng thời nêu rõ những giới hạn còn lại về xác thực, đồng bộ nhiều thiết bị và quyền biểu diễn âm nhạc công cộng.

Từ khóa: Max-Heap; hàng đợi ưu tiên; danh sách liên kết đôi vòng; Hash Map; chống spam; React; PWA; hệ thống phát nhạc cộng đồng.

ABSTRACT

Community music-selection systems must efficiently identify the highest-priority track, support bidirectional playback history, and limit repeated requests from a single user. This report presents **HeapBeat**, a web/PWA prototype for self-study rooms. A Max-Heap with an index map manages the voting queue, a circular doubly linked list models playback history, and a StudentID-indexed hash map enforces a sliding-window anti-spam policy. React 19 and TypeScript implement the client, while an authoritative C11 backend owns the core data structures and exposes their state through a JSON API.

The evaluation combines invariant checking, asymptotic complexity analysis, 22 automated unit tests, warning-free C compilation, web build verification, and interface checks at desktop, narrow-laptop, and mobile widths. All 22 tests pass. Heap peek, playlist Next/Previous, and average hash lookup operate in $O(1)$; insert, extract, arbitrary removal, and vote updates operate in $O(\log n)$. At the inspected viewports, the queue introduces no horizontal overflow. The results support the suitability of the selected data structures for the stated use case, while remaining limited to the scale and conditions of an academic prototype.

Keywords: Max-Heap; priority queue; circular doubly linked list; hash map; anti-spam; React; PWA; community music system.

Mục lục

Tóm tắt	i
Abstract	iii
Danh mục hình	ix
Danh mục bảng	xi
Danh mục từ viết tắt và ký hiệu	xiii
1 Giới thiệu và đặc tả nghiên cứu	1
1.1 Bài toán	1
1.2 Mục tiêu và phương pháp	1
1.3 Câu hỏi nghiên cứu	1
1.4 Đóng góp chính	2
1.5 Giả thuyết và thiết kế đánh giá	2
1.6 Ánh xạ yêu cầu chức năng và cấu trúc dữ liệu	3
1.7 Tiêu chí nghiệm thu bản nộp	3
2 Cơ sở lý thuyết	5
2.1 Max-Heap cho hàng đợi ưu tiên	5
2.2 Danh sách liên kết đôi vòng cho lịch sử phát	5
2.3 Hash Map và cửa sổ trượt chống gửi quá mức	5
3 Mô hình hoạt động	7
3.1 Kiến trúc tổng thể	7
3.2 Luồng yêu cầu, bình chọn và phát bài	7
3.3 Ranh giới vai trò	7
4 Thiết kế thuật toán	9
4.1 Thay đổi bình chọn trong Max-Heap	9
4.2 Phát bài kế tiếp	9
4.3 Chống gửi quá mức và thu hồi yêu cầu	9

4.4	Tổ chức mã nguồn	10
5	Cài đặt web/PWA và bản quyền	11
5.1	Lựa chọn công nghệ	11
5.2	Kiểm soát quyền phát	11
5.3	Chiến lược không vi phạm bản quyền	11
5.4	Mức độ đa nền tảng đã kiểm tra	12
6	Kết quả sản phẩm	13
7	Kiểm thử và kết quả	15
7.1	Thiết kế kiểm tra và tiêu chí đánh giá	15
7.2	Kiểm thử đơn vị cho cấu trúc dữ liệu	15
7.3	Tạo bản dựng và kiểm tra tích hợp trên trình duyệt	15
8	Phân tích và đánh giá	17
8.1	Trả lời câu hỏi nghiên cứu	17
8.2	So sánh lựa chọn hàng đợi ưu tiên	17
8.3	Điểm mạnh	17
8.4	Hạn chế và rủi ro còn lại	18
8.5	Mối đe dọa đến độ tin cậy	18
9	Kết luận và hướng phát triển	19
9.1	Hướng phát triển ưu tiên	19
	Tài liệu tham khảo	21
A	Các thuật toán trọng tâm và giải thích từng dòng	23
A.1	Bộ so sánh đa tiêu chí	24
A.2	Thêm phần tử và <code>heapifyUp</code>	25
A.3	Thay đổi bình chọn và chọn hướng tái cân bằng	26
A.4	<code>ExtractMax</code>	27
A.5	Xóa một yêu cầu bất kỳ	28
A.6	Danh sách liên kết đôi vòng	29
A.7	Cửa sổ trượt, chặn và thu hồi	30
A.8	Chọn bài kế tiếp và phối hợp hai CTDL	31
B	Ma trận truy vết yêu cầu	33
C	Đặc tả bất biến và điều kiện kiểm chứng	35

C.1	Max-Heap	35
C.2	Danh sách liên kết đôi vòng	35
C.3	SpamGuard	35
C.4	Tính nguyên tử của trạng thái backend	36
D	Hướng dẫn tái lập kết quả	37
D.1	Môi trường tối thiểu	37
D.2	Quy trình và đầu ra kỳ vọng	37
D.3	Danh sách xác nhận độc lập	38
E	Kế hoạch thực hiện và phân công nhiệm vụ	39
E.1	Ma trận phân công cân bằng	39
E.2	Biểu đồ Gantt	41
E.3	Mốc kiểm soát và tiêu chí hoàn thành	42
E.4	Quy tắc bàn giao và kiểm tra chéo	42

Danh sách hình vẽ

6.1	Giao diện HeapBeat tại 1600x1000 với danh mục piano và Max-Heap gồm 4 phần tử.	13
-----	--	----

Danh sách bảng

1.1	Giả thuyết, biên quan sát và tiêu chí chấp nhận.	2
1.2	Ánh xạ giữa yêu cầu chức năng, cấu trúc dữ liệu và hành vi chương trình. .	3
1.3	Tổng hợp tiêu chí nghiệm thu bản nộp.	3
7.1	Môi trường đánh giá và tiêu chí xác nhận kết quả.	15
8.1	Kết quả trả lời ba câu hỏi nghiên cứu.	17
A.1	Danh mục thuật toán được phân tích chi tiết.	23
B.1	Truy vết từ yêu cầu đến mã nguồn và kết quả kiểm tra.	33
C.1	Điều kiện kiểm chứng và dấu hiệu vi phạm bất biến.	36
D.1	Môi trường tối thiểu để tái lập các nhóm kết quả.	37
E.1	Ma trận phân công cân bằng theo điểm công việc và kiểm tra chéo.	40
E.2	Các mốc kiểm soát dùng để quyết định chuyển giai đoạn.	42

DANH MỤC TỪ VIẾT TẮT VÀ KÝ HIỆU

Ký ngữ	hiệu/thuật	Diễn giải trong báo cáo
H, n		Mảng lưu Heap và số phần tử hiện hành.
$i, parent(i)$		Chỉ số một node và chỉ số node cha trong Heap.
CDLL		Circular Doubly Linked List – danh sách liên kết đôi vòng.
RQ		Research Question – câu hỏi nghiên cứu.
Điều kiện kiểm chứng		Tiêu chí độc lập dùng để xác định một ca kiểm thử đúng hay sai.
PWA		Progressive Web App – ứng dụng web có manifest và service worker.
TB $O(1)$		Độ phức tạp trung bình hằng số của thao tác Hash Map.

CHƯƠNG 1. GIỚI THIỆU VÀ ĐẶC TẢ NGHIÊN CỨU

HeapBeat chuyển một bài toán bình chọn quen thuộc thành mô hình trực quan cho ba cấu trúc dữ liệu trọng tâm: hàng đợi ưu tiên, danh sách liên kết vòng và bảng băm.

1.1 Bài toán

Tại các không gian công cộng của trường như sảnh hoặc phòng tự học, nhiều sinh viên cùng đề xuất bài hát. Danh sách phát phải phản ánh lựa chọn của cộng đồng, vẫn cho phép Next/Prev/Repeat, và không bị một tài khoản chi phối. HeapBeat cung cấp hai vai trò: **sinh viên** gửi yêu cầu và bình chọn; **quản trị viên** điều khiển trình phát, kiểm duyệt bản quyền, cấu hình phòng và theo dõi sự cố.

1.2 Mục tiêu và phương pháp

Mục tiêu của nghiên cứu là đánh giá mức độ phù hợp của ba cấu trúc dữ liệu đối với một luồng chọn nhạc cộng đồng, thay vì chứng minh khả năng chịu tải của một dịch vụ thương mại. Quá trình thực hiện gồm bốn bước: đặc tả yêu cầu và bất biến; xây dựng lõi cấu trúc dữ liệu độc lập; tích hợp lõi vào ứng dụng; và kiểm tra kết quả bằng các điều kiện đã xác định trước. Mỗi câu hỏi nghiên cứu được đối chiếu với ít nhất một nhóm kiểm thử đơn vị và một hành vi có thể quan sát trên giao diện.

Dữ liệu đầu vào bao gồm thứ tự gửi yêu cầu, điểm số, lịch sử bình chọn, thời điểm gửi và trạng thái của mã sinh viên. Các đại lượng được theo dõi gồm phần tử đầu Heap, thứ tự xếp hạng, con trỏ lịch sử phát, danh sách yêu cầu bị thu hồi và mã phản hồi của hệ thống. Bộ dữ liệu minh họa được giữ cố định để các lần chạy có thể lặp lại. Vì chưa thực hiện phép đo tải quy mô lớn, những nhận định về hiệu năng trong báo cáo chỉ dựa trên phân tích tiệm cận.

1.3 Câu hỏi nghiên cứu

RQ1. Max-Heap có bản đồ chỉ số có duy trì đúng thứ tự ưu tiên sau khi thêm, thay đổi bình chọn, xóa và lấy phần tử đầu, đồng thời giữ chi phí cập nhật $O(\log n)$ hay không?

RQ2. Danh sách liên kết đôi vòng có mô hình hóa tự nhiên Next, Previous và Repeat All với chi phí $O(1)$ và xử lý đúng các trường hợp rỗng hoặc một phần tử hay không?

RQ3. Hash Map theo StudentID có thể phát hiện gửi trùng, vượt hạn mức và cung cấp danh sách yêu cầu cần thu hồi mà không làm phá vỡ bất biến của Heap hay không?

1.4 Đóng góp chính

1. Một lõi CTDL độc lập giao diện, có bất biến, bộ so sánh và quy tắc phân giải đồng hạng được đặc tả rõ.
2. Một ứng dụng hoàn chỉnh từ khâu gửi yêu cầu đến phát bài, qua đó quan sát được tác động của bình chọn, lấy phần tử đầu Heap và chặn tài khoản vi phạm.
3. Bộ kiểm thử có thể tái lập, bộ kiểm thử backend C và ma trận liên kết yêu cầu với phần hiện thực và kết quả kiểm tra.
4. Quy trình kiểm duyệt nguồn nhạc tách thông tin mô tả khỏi hồ sơ chứng minh quyền phát.

1.5 Giả thuyết và thiết kế đánh giá

Ba giả thuyết được xác lập trước khi tiến hành kiểm tra. Biến tác động gồm chuỗi thao tác, dữ liệu bình chọn, thời điểm gửi yêu cầu và trạng thái của lịch sử phát. Biến quan sát gồm phần tử đầu Heap, ánh xạ chỉ số, liên kết giữa các node, tập ID bị thu hồi và kích thước vùng hiển thị. Bảng 1.1 quy định trước tiêu chí chấp nhận cho từng giả thuyết, nhờ đó kết luận không phụ thuộc vào việc lựa chọn kết quả sau khi chạy chương trình.

ID	Giả thuyết	Biến quan sát	Tiêu chí chấp nhận
H1	Các thao tác thêm, bình chọn, xóa và lấy phần tử đầu không phá vỡ thứ tự Heap hoặc ánh xạ chỉ số.	Phần tử đầu, bộ so sánh, <code>requestId -> index</code> .	Bất biến đúng sau từng thao tác; 7/7 phép kiểm tra đạt.
H2	CDLL hỗ trợ Next/Prev/Repeat mà không cần xử lý chỉ số biên.	<code>head/tail/current</code> , liên kết <code>next/prev</code> .	Trường hợp rỗng, một node và khép vòng hai chiều đều đúng; 4/4 phép kiểm tra đạt.
H3	SpamGuard phát hiện yêu cầu trùng hoặc vượt giới hạn và thu hồi đúng các yêu cầu đang chờ.	Trạng thái chặn, cửa sổ thời gian, <code>purgeIds</code> .	Yêu cầu của tài khoản bị chặn không được đưa vào Heap; 5/5 phép kiểm tra đạt.

Bảng 1.1. Giả thuyết, biến quan sát và tiêu chí chấp nhận.

1.6 Ánh xạ yêu cầu chức năng và cấu trúc dữ liệu

Yêu cầu	CTDL	Cách hiện thực trong HeapBeat	Mục tiêu
Xếp bài theo bình chọn tức thời	Max-Heap + Map chỉ số	Gốc Heap luôn là bài tiếp theo; Upvote và Downvote gọi <code>heapifyUp/Down</code> .	$O(\log n)$
Next, Prev, Repeat và thêm cuối	Danh sách liên kết đôi vòng	<code>head.prev</code> là đuôi; <code>tail.next=head</code> ; <code>current</code> đi hai chiều và <code>addLast</code> thêm bài cuối danh sách.	$O(1)$
Chống spam và phá hoại	Hash Map theo StudentID	Cửa sổ trượt 10 phút; yêu cầu thứ tư hoặc gửi trùng thì chặn 30 phút và thu hồi toàn bộ bài đang chờ.	TB $O(1)$
An toàn bản quyền	Kiểm tra giấy phép	Chỉ bài <code>approved</code> , có hồ sơ giấy phép và <code>publicPlaybackAllowed</code> mới được yêu cầu.	Khóa mặc định

Bảng 1.2. Ánh xạ giữa yêu cầu chức năng, cấu trúc dữ liệu và hành vi chương trình.

1.7 Tiêu chí nghiệm thu bản nộp

3

cấu trúc dữ liệu

22/22

kiểm thử đơn vị

18

file piano MP3

Hạng mục	Tiêu chí nghiệm thu	Kết quả
Đúng dẫn CTDL	Toàn bộ kiểm thử và bất biến sau thao tác đều đúng.	22/22 đạt
Mã đối chiếu C	Không có cảnh báo khi dùng <code>-Werror</code> ; mọi lệnh <code>assert</code> đúng.	Đạt
Khả năng đóng gói	Kiểm tra kiểu TypeScript và tạo bản dựng web với mã thoát 0.	Đạt
Giao diện thích ứng	Thân trang và hàng đợi không cuộn ngang tại các kích thước đã công bố.	Đạt
Nguồn âm thanh	Các file cục bộ có thông tin ghi công và được kiểm tra quyền trước khi phát.	18/18 hồ sơ

Bảng 1.3. Tổng hợp tiêu chí nghiệm thu bản nộp.

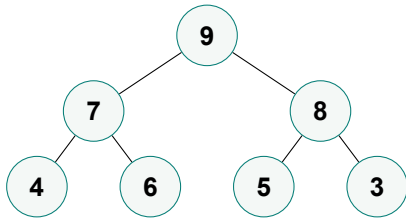
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1 Max-Heap cho hàng đợi ưu tiên

Max-Heap là cây nhị phân gần đầy, lưu bằng mảng $H[0..n-1]$ [1]. Với nút i :

$$\text{parent}(i) = \left\lfloor \frac{i-1}{2} \right\rfloor, \quad \text{left}(i) = 2i+1, \quad \text{right}(i) = 2i+2.$$

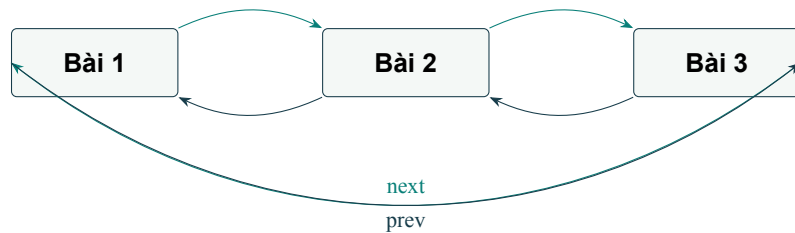
Bất biến cần giữ là $\text{priority}(H[\text{parent}(i)]) \geq \text{priority}(H[i])$. HeapBeat so sánh theo **điểm giảm dần**, tiếp theo là số lượt tăng bình chọn giảm dần, rồi thời điểm yêu cầu tăng dần (FIFO). Bản đồ `requestId` $\rightarrow$ `index` giúp tìm phần tử cần đổi bình chọn trong $O(1)$ trung bình.



Khi tăng điểm: cập nhật điểm, so sánh với node cha và đổi chỗ cho đến khi bất biến được phục hồi. Khi giảm điểm hoặc lấy node gốc, phần tử được so với hai node con và đi xuống nhánh có mức ưu tiên lớn hơn.

2.2 Danh sách liên kết đôi vòng cho lịch sử phát

Mỗi node chứa `prev`, `next` và một bài hát. Nút cuối nối về đầu, đầu nối lùi về cuối nên chế độ lặp toàn bộ là thuộc tính tự nhiên của cấu trúc, không cần xử lý biên riêng [2].



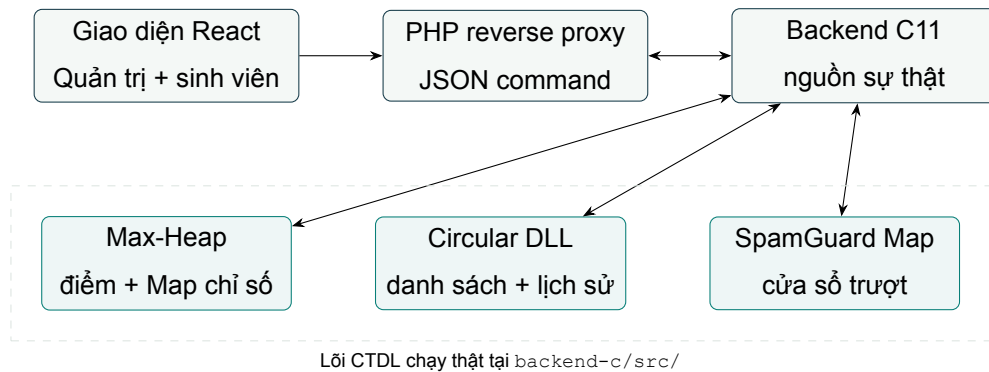
2.3 Hash Map và cửa sổ trượt chống gửi quá mức

Khóa là StudentID đã chuẩn hóa; giá trị lưu các bài được yêu cầu gần đây, các yêu cầu đang sở hữu, `blockedUntil`, lý do và số lần vi phạm. Trước mỗi lần gửi, hệ thống bỏ các mốc cũ hơn 10 phút, kiểm tra trạng thái chặn, bài trùng và hạn mức. Tra cứu trung bình $O(1)$; số mốc cần duyệt bị chặn trên bởi cấu hình nhỏ.

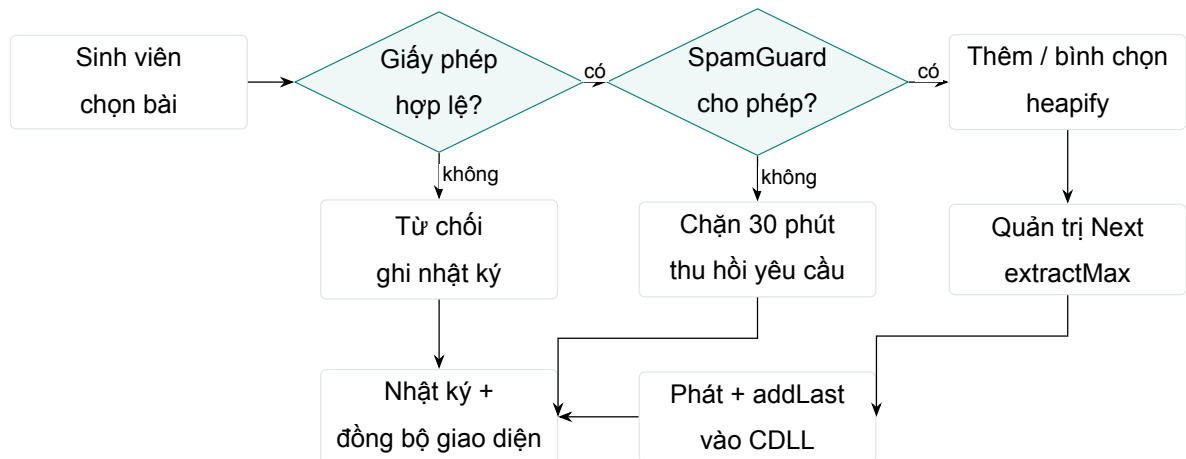
Thao tác	Xem gốc Heap	Thêm vào Heap	Bình chọn	CDLL Next/Prev	Tra Hash Map	Thu hồi k bài
Độ phức tạp	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$	TB $O(1)$	$O(k \log n)$

CHƯƠNG 3. MÔ HÌNH HOẠT ĐỘNG

3.1 Kiến trúc tổng thể



3.2 Luồng yêu cầu, bình chọn và phát bài



3.3 Ranh giới vai trò

	Sinh viên	Quản trị viên
Hàng đợi	Gửi yêu cầu, tăng/giảm bình chọn bằng tài khoản đăng nhập	Xóa, trộn và lấy bài ở gốc Heap
Trình phát	Theo dõi trạng thái	Phát/tạm dừng, Next/Prev, tua, âm lượng và lặp
Quản trị	Xem bài của mình và trạng thái phản hồi	Tạo phòng, cấu hình chống gửi quá mức, duyệt giấy phép, bỏ chặn và xuất dữ liệu

Lưu ý về bảo mật: cơ chế đăng nhập của bản đồ án chỉ phục vụ minh họa và được xử lý ở phía trình duyệt; không thay thế đăng nhập một lần của nhà trường, xác thực máy chủ hoặc băm mật khẩu trong một hệ thống triển khai thực tế.

CHƯƠNG 4. THIẾT KẾ THUẬT TOÁN

Các đoạn mã dưới đây được rút gọn theo cú pháp C để làm rõ thao tác trên cấu trúc dữ liệu. Phần hiện thực chính thức nằm trong `backend-c/src/`; giao diện gọi các thao tác qua adapter `src/lib/c-backend.ts`.

4.1 Thay đổi bình chọn trong Max-Heap

```
void change_vote(MaxHeap *h, int id,
    int delta) {
    int i = h->index_by_id[id];
    h->items[i].score += delta;
    if (delta > 0) heapify_up(h, i);
    else heapify_down(h, i);
}
```

Bất biến:

- Mỗi lần đổi chỗ phải cập nhật hai vị trí trong `indexByRequestId`.
- Một sinh viên chỉ có một trạng thái bình chọn trên mỗi yêu cầu.
- Trường hợp bằng điểm được phân giải bằng lượt tăng bình chọn, `shuffleOrder` và `requestedAt`.

```
QueueItem item = heap_extract_max(&
    heap);
playlist_add_last(&playlist, item.
    song_id);
playlist.current = playlist.head->prev
    ; /* tail */

/* Repeat All */
playlist.current = playlist.current->
    next;
```

Bất biến:

- Bài lấy từ hàng đợi luôn là gốc Heap tại thời điểm thực hiện.
- Bài đã phát được nối cuối CDLL, không mất lịch sử.
- `next/prev` không tạo chỉ số nằm ngoài miền hợp lệ.

4.3 Chống gửi quá mức và thu hồi yêu cầu

```
StudentState *s = map_get_or_create(&guard, student_id);
prune_old_requests(s, now - 10 * 60);
if (s->blocked_until > now) return REJECTED;
if (has_duplicate(s, song_id) || s->recent_count >= 3) {
    s->blocked_until = now + 30 * 60;
    heap_remove_many(&heap, s->active_request_ids);
    return BLOCKED;
}
return ALLOWED;
```

Tình huống biên	Cách xử lý	Kết quả
Heap hoặc danh sách phát rỗng	Trả <code>null</code> ; giao diện dừng an toàn và hiển thị phản hồi.	Không lỗi
Một node trong CDLL	<code>next</code> và <code>prev</code> cùng trỏ về node đó.	Repeat hợp lệ
Lặp lại cùng một bình chọn	Không đổi điểm, trả <code>delta=0</code> .	Không nhân đôi bình chọn
Xóa phần tử giữa heap	Đưa phần tử cuối vào vị trí trống, heapify cả hai hướng.	Giữ bất biến
Sinh viên đang bị chặn	Từ chối trước khi thay đổi Heap; hiển thị thời gian còn lại.	Dừng sớm
Bài thiếu giấy phép	Nút yêu cầu bị vô hiệu hóa; backend C chỉ nhận mã bài thuộc danh mục cố định.	Dùng an toàn

4.4 Tổ chức mã nguồn

```
src/lib/c-backend.ts # typed adapter for C snapshots
src/app/model.ts # UI state and C snapshot reducer
src/audio/useAudioPlayer.ts # HTML5 MP3 player
src/components/ # player, queue, catalog, modals
src/App.tsx # application orchestration
tests/heapbeat.test.ts # 22 unit tests
backend-c/src/ # authoritative C11 algorithms + HTTP
backend-c/tests/ # 56 C assertions for invariants and API flow
public/api.php # PHP to C11 reverse proxy
```

CHƯƠNG 5. CÀI ĐẶT WEB/PWA VÀ BẢN QUYỀN

5.1 Lựa chọn công nghệ

Tầng	Công nghệ	Lý do
Giao diện	React 19, TypeScript, CSS	Tách thành phần theo chức năng, kiểm tra kiểu tĩnh và hỗ trợ nhiều kích thước màn hình [3].
Lỗi thuật toán	Backend C11	Thực thi Max-Heap, CDLL và Hash Map độc lập với giao diện.
Đa nền tảng	Vite, Web App Manifest + Service Worker	Cho phép cài dưới dạng PWA trên máy tính và điện thoại từ cùng một bản dựng web [4].
Cổng API	PHP reverse proxy	Chuyển JSON tới tiến trình C nội bộ, không giữ bản sao state.
Âm thanh minh họa	HTML5 Audio	Phát 18 file piano MP3 cục bộ, không phụ thuộc dịch vụ truyền phát bên ngoài [5].

5.2 Kiểm soát quyền phát

Mỗi bản ghi bài hát lưu nguồn, loại và URL giấy phép, thông tin ghi công, trạng thái duyệt và cờ cho phép phát công cộng. Một bài chỉ được đưa vào hàng đợi khi đủ cả ba điều kiện:

$$approved \wedge publicPlaybackAllowed \wedge licenseUrl \neq \emptyset.$$

Giao diện vô hiệu hóa nút yêu cầu ngay khi thiếu điều kiện. Backend C chỉ chấp nhận `songId` nằm trong danh mục cố định đã được đóng gói; mọi mã không hợp lệ bị từ chối trước khi Heap thay đổi. Quản trị viên có màn hình kiểm duyệt và thông tin nguồn để xem lại căn cứ của từng quyết định.

5.3 Chiến lược không vi phạm bản quyền

1. Gói phát hành chứa 18 bản piano do chủ dự án cung cấp để minh họa nội bộ. Danh mục lưu thông tin ghi công từ metadata ID3; mỗi file phải được xác minh quyền trước khi triển khai công cộng.

2. Openverse hoặc Jamendo chỉ là hướng mở rộng để *tìm* nội dung [7]. Metadata API không tự động chứng minh quyền phát tại không gian công cộng; quản trị viên vẫn phải mở trang nguồn và lưu hồ sơ kiểm tra.
3. Không tải, lưu đệm, chuyển đổi hay phát tập trung nội dung từ Spotify, Apple Music hoặc YouTube. Chính sách Spotify nêu dịch vụ dành cho sử dụng cá nhân, không thương mại và không được phát công khai trong mô hình doanh nghiệp [8].
4. Với CC BY, phải ghi công, dẫn link giấy phép và nêu thay đổi nếu có. Với triển khai thật, trường cần kiểm tra thêm quyền biểu diễn công cộng theo pháp luật áp dụng [6].

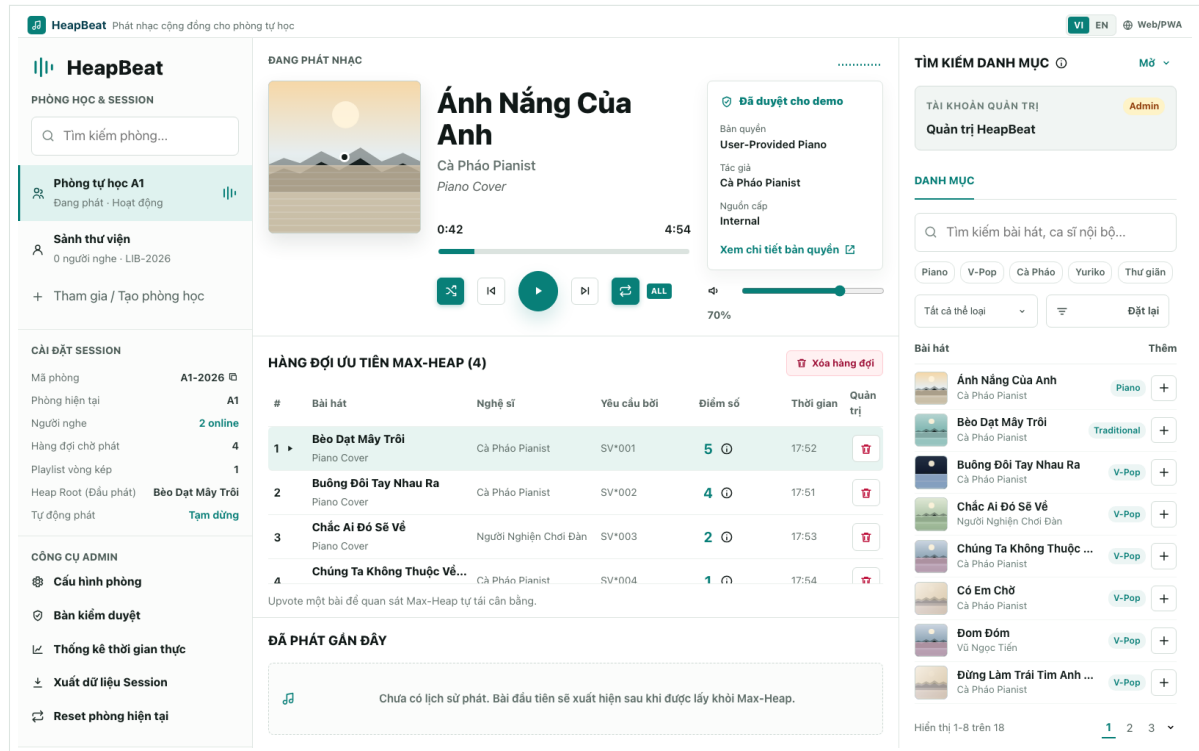
Nguồn chính thức đã đối chiếu (07/2026): Openverse API; Jamendo API; Creative Commons licenses; Spotify Developer Policy.

5.4 Mức độ đa nền tảng đã kiểm tra

- Bản dựng web đã chạy qua máy chủ HTTP; manifest và service worker được đóng gói đầy đủ.
- Giao diện đã được kiểm tra ở kích thước máy tính 1600x1000 và điện thoại 390x844.
- Cùng một mã nguồn hoạt động trên Windows, macOS, Linux, iOS và Android qua trình duyệt. Tính năng thêm vào màn hình chính và tự động phát vẫn cần kiểm tra thêm trên thiết bị thật.

CHƯƠNG 6. KẾT QUẢ SẢN PHẨM

Màn hình quản trị được chia thành ba vùng: danh sách phòng ở bên trái, trình phát và hàng đợi ở giữa, danh mục bài hát ở bên phải. Phần tử đầu Heap, điểm số và lịch sử phát được hiển thị ngay trên giao diện để người đọc có thể đối chiếu với phần thuật toán.



Hình 6.1. Giao diện HeapBeat tại 1600x1000 với danh mục piano và Max-Heap gồm 4 phần tử.

Ngoài ba yêu cầu cấu trúc dữ liệu cốt lõi, sản phẩm còn hỗ trợ phân quyền sinh viên/quản trị viên, xuất phiên làm việc dưới dạng JSON, giao diện song ngữ, phím tắt điều khiển, lưu trạng thái phiên, kiểm duyệt giấy phép và đồng bộ tùy chọn trong mạng nội bộ.

Bề mặt	Kết quả hiện thực	CTDL/logic quan sát được
Trình phát	Phát/tạm dừng, Next/Prev, tua, âm lượng, các chế độ lặp và thông tin ghi công.	Con trỏ current/next/prev của CDLL
Hàng đợi ưu tiên	Làm nổi bật gốc Heap, điểm, thời điểm, bình chọn và thao tác xóa; thứ tự đổi ngay sau khi bình chọn.	Max-Heap + Map chỉ số
Danh mục bài hát	Tìm kiếm, lọc thể loại, phân trang và chỉ cho phép yêu cầu bài hợp lệ.	Kiểm tra giấy phép + thêm vào Heap
Kiểm duyệt	Duyệt hoặc từ chối giấy phép, xem tài khoản bị chặn và bỏ chặn thủ công.	SpamGuard Hash Map
Phòng và thống kê	Tạo hoặc chuyển phòng độc lập, theo dõi kết nối và thống kê điểm, lịch sử, vi phạm.	Trạng thái phòng + nhật ký

CHƯƠNG 7. KIỂM THỬ VÀ KẾT QUẢ

7.1 Thiết kế kiểm tra và tiêu chí đánh giá

Mỗi phép kiểm tra bắt đầu từ một trạng thái dữ liệu xác định. Các kiểm thử đơn vị gọi trực tiếp lớp cấu trúc dữ liệu; chương trình C sử dụng `assert`; bản dựng TypeScript được kiểm tra kiểu tĩnh trước khi đóng gói. Một trường hợp chỉ được công nhận khi cả kết quả trả về lẫn bất biến sau thao tác đều đúng. Đối với giao diện thích ứng, điều kiện chấp nhận là `scrollWidth = clientWidth` cho thân trang và vùng hàng đợi, nội dung chính không bị cắt và các điều khiển vẫn có nhãn hỗ trợ truy cập.

Thành phần	Môi trường/lệnh	Điều kiện kiểm chứng
TypeScript CTDL	Node.js 20+, Vitest; npm run test	Giá trị trả về, thứ tự gốc Heap, liên kết vòng và trạng thái SpamGuard
Mã đối chiếu C	C11, -Wall -Wextra -Werror	Tất cả <code>assert</code> đạt, không cảnh báo biên dịch
Bản dựng ứng dụng	TypeScript + Vite; npm run build	Hoàn tất bản dựng, 38 module được xử lý
Giao diện thích ứng	Trình duyệt hiện đại; 1400x900, 1200x820, 390x844	Không tràn thân trang hoặc hàng đợi; bảng thu gọn cột và chuyển thành thẻ ở kích thước điện thoại

Bảng 7.1. Môi trường đánh giá và tiêu chí xác nhận kết quả.

7.2 Kiểm thử đơn vị cho cấu trúc dữ liệu

Lệnh `npm run test` thực hiện 22 phép kiểm tra bằng Vitest. Các phép kiểm tra tác động trực tiếp lên lớp cấu trúc dữ liệu, trạng thái phát và danh mục MP3, không phụ thuộc vào ảnh chụp màn hình hoặc thao tác thủ công.

7.3 Tạo bản dựng và kiểm tra tích hợp trên trình duyệt

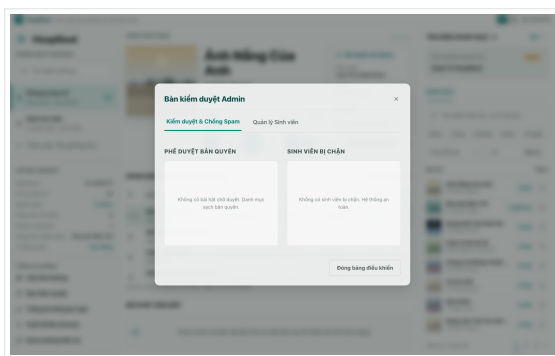
- `npm run format:check`: toàn bộ TypeScript, CSS và script đúng định dạng.
- `npm run build`: TypeScript và Vite tạo bản dựng hoàn chỉnh; 38 module được xử lý.
- Bộ kiểm thử C: chạy `make test` trong thư mục `backend-c`; GCC/Clang dùng `-Wall -Wextra -Werror` và cả 56 assertion đều đạt.
- Trình duyệt: tiêu đề và URL đúng; trang hiển thị đầy đủ; không có lỗi hoặc cảnh báo trên console liên quan đến ứng dụng.

Nhóm	Ca kiểm thử tiêu biểu	Kết quả
Circular DLL (4)	Danh sách rỗng; vòng tail→head; vòng head→tail; addLast không phá liên kết.	4/4 đạt
Max-Heap (7)	Gốc có ưu tiên cao nhất; heapify hai hướng; thứ tự FIFO khi bằng điểm; lấy, xóa và gỡ bình chọn của người bị chặn.	7/7 đạt
SpamGuard (5)	Cho phép 3 yêu cầu; chặn yêu cầu thứ tư; phát hiện bài trùng; thu hồi bài; loại mốc cũ và hết thời hạn chặn.	5/5 đạt
Trạng thái phát (4)	Phòng rỗng; xóa hàng đợi; từ chối bài đang phát; sửa vị trí phát lưu sai.	4/4 đạt
Danh mục MP3 (2)	Đủ 18 file, ID và đường dẫn không trùng; URL hoạt động khi triển khai trong thư mục con.	2/2 đạt

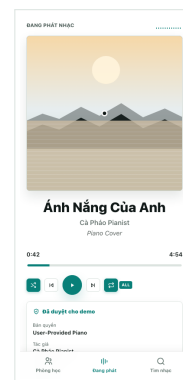
- Các tình huống đã thực hiện gồm: quản trị viên đăng nhập; Next lấy gốc Heap làm hàng đợi giảm từ 4 xuống 3; Prev quay lại đúng node trong CDLL; duyệt giấy phép làm nút yêu cầu khả dụng; bình chọn làm điểm tăng từ 4 lên 5 và thay đổi gốc Heap; tài khoản SV9999 gửi trùng bị chặn và các bài liên quan được thu hồi; phòng mới hoạt động độc lập; giao diện chuyển đúng sang bố cục điện thoại.

Tại kích thước máy tính hẹp 1200x820, cột giữa rộng 660 px và hàng đợi có `clientWidth = scrollWidth = 591 px`; ở mobile 390x844, body có `clientWidth = scrollWidth = 375 px` và hàng đợi có `clientWidth = scrollWidth = 345 px`. Đây là bằng chứng đo trực tiếp cho việc loại bỏ cuộn ngang từng xuất hiện ở bảng 7 cột. Ở mức hẹp, nghệ sĩ và thời gian được đưa vào dòng thông tin phụ của bài; ở điện thoại, mỗi yêu cầu chuyển thành thẻ hai hàng nhưng vẫn giữ thứ hạng, người yêu cầu, điểm và thao tác quản trị.

Các trường hợp biên cũng được đưa vào bộ kiểm thử: xóa hết hàng đợi không làm mất bài đang phát; danh sách phát rỗng luôn đưa vị trí phát về `null/0`; và dữ liệu danh mục cũ không thể ghi đè danh mục piano sau khi nâng cấp.



Bàn kiểm duyệt: danh mục đã duyệt và không có tài khoản bị chặn.



Điện thoại 390x844: trình phát không tràn ngang.

CHƯƠNG 8. PHÂN TÍCH VÀ ĐÁNH GIÁ

8.1 Trả lời câu hỏi nghiên cứu

RQ	Kết quả kiểm tra	Kết luận trong phạm vi đề tài
RQ1	7/7 phép kiểm tra Heap; bộ so sánh và Map chỉ số được kiểm tra sau thao tác thêm, bình chọn, xóa và lấy gốc.	Max-Heap giữ đúng phần tử gốc, quy tắc phân giải khi bằng điểm và chi phí cập nhật $O(\log n)$.
RQ2	4/4 phép kiểm tra CDLL cho danh sách rỗng, một node và khép vòng hai chiều.	Next/Prev/Repeat có chi phí $O(1)$ và không cần xử lý chỉ số biên.
RQ3	5/5 phép kiểm tra SpamGuard cùng phép kiểm tra thu hồi khỏi Heap.	Tra cứu trung bình $O(1)$, phát hiện yêu cầu trùng hoặc vượt giới hạn và trả đúng các ID cần thu hồi.

Bảng 8.1. Kết quả trả lời ba câu hỏi nghiên cứu.

8.2 So sánh lựa chọn hàng đợi ưu tiên

Phương án	Xem cực đại	Thêm	Đổi bình chọn	Đánh giá
Mảng chưa sắp xếp	$O(n)$	$O(1)$	$O(1)$	Khi phát bài phải quét toàn bộ mảng.
Mảng luôn sắp xếp	$O(1)$	$O(n)$	$O(n)$	Bình chọn liên tục làm dịch chuyển nhiều phần tử.
Cây cân bằng	$O(\log n)$	$O(\log n)$	$O(\log n)$	Đáp ứng yêu cầu nhưng làm phân hiện thực phức tạp hơn.
Max-Heap + Map	$O(1)$	$O(\log n)$	$O(\log n)$	Cân bằng giữa hiệu năng, bộ nhớ và khả năng minh họa.

8.3 Điểm mạnh

- Ba cấu trúc dữ liệu tương ứng rõ với ba yêu cầu cốt lõi; lớp thuật toán không phụ thuộc vào React nên có thể kiểm tra riêng.
- Map chỉ số giúp thay đổi bình chọn mà không phải quét tìm yêu cầu; quy tắc phân giải khi bằng điểm được xác định và giữ ổn định.

- SpamGuard không chỉ từ chối yêu cầu mới mà còn trả về các ID để thu hồi toàn bộ bài đang chờ của tài khoản vi phạm.
- Giao diện lọc theo hồ sơ giấy phép; backend C chỉ nhận mã bài thuộc danh mục cố định; 18 file piano đều có thông tin ghi công và mô tả phạm vi sử dụng.
- Giao diện thể hiện rõ gốc Heap, điểm số và vòng lịch sử phát, tạo điều kiện đối chiếu giữa lý thuyết và hành vi chương trình.

8.4 Hạn chế và rủi ro còn lại

- Backend C hiện lưu state trong RAM; cần bổ sung cơ sở dữ liệu và cơ chế phục hồi khi triển khai lâu dài hoặc vận hành nhiều phòng.
- Tài khoản minh họa được lưu ở phía trình duyệt; hệ thống chưa có SSO, bấm mật khẩu tại máy chủ, token, giới hạn theo IP hoặc kiểm soát quyền ở máy chủ.
- Số người nghe hiện được ước lượng bằng số tab gửi tín hiệu định kỳ, không phải số người dùng duy nhất đã xác thực.
- Chưa đo tải với hàng nghìn yêu cầu và chưa kiểm thử PWA trên nhiều thiết bị thật.
- Quyền biểu diễn công cộng phụ thuộc địa điểm và giấy phép cụ thể; phần mềm không thể tự suy ra quyền pháp lý chỉ từ một URL.

8.5 Mối đe dọa đến độ tin cậy

- **Độ tin cậy nội tại:** bộ kiểm thử được xây dựng từ cùng đặc tả với chương trình nên có thể lặp lại một giả định sai. Chương trình C độc lập và việc kiểm tra trên giao diện làm giảm nhưng không loại bỏ hoàn toàn rủi ro này.
- **Khả năng khái quát:** dữ liệu chỉ gồm 18 bài và hàng đợi nhỏ; chưa thể suy ra độ trễ khi có hàng nghìn yêu cầu hoặc nhiều phòng cập nhật đồng thời.
- **Độ tin cậy đo lường:** số người nghe hiện được suy ra từ tín hiệu của tab; phép đo kích thước DOM chưa thay thế đánh giá khả dụng với người dùng thật.
- **Độ tin cậy pháp lý:** metadata và URL giấy phép là bằng chứng kỹ thuật, không tự động xác lập quyền biểu diễn công cộng tại mọi địa điểm.

CHƯƠNG 9. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Kết quả cho thấy bản nộ HeapBeat đáp ứng các yêu cầu đã đặt ra. Max-Heap đưa bài có mức ưu tiên cao nhất lên đầu hàng đợi; danh sách liên kết đôi vòng hỗ trợ Next/Prev/Repeat; Hash Map kiểm soát tần suất gửi yêu cầu theo StudentID. Các thao tác chính có chi phí $O(\log n)$ hoặc $O(1)$ trung bình. Nhận định này được đối chiếu bằng 22 kiểm thử đơn vị, chương trình C, bản dựng web và các tình huống sử dụng trên máy tính lẫn điện thoại.

9.1 Hướng phát triển ưu tiên

1. Chuyển trạng thái sang PostgreSQL/Redis và WebSocket để nhiều thiết bị cập nhật đồng thời.
2. Tích hợp SSO của trường, xác thực tại máy chủ, RBAC, băm mật khẩu và nhật ký bất biến.
3. Bổ sung nhập dữ liệu từ Openverse/Jamendo theo quy trình tìm kiếm, chờ duyệt và lưu hồ sơ; không tự động phát ngay sau khi nhập.
4. Kiểm thử PWA trên Windows, macOS, Android và iOS; bổ sung icon PNG cho từng thiết bị.
5. Bổ sung cơ chế giảm trọng số bình chọn theo thời gian, phát hiện thông đồng và đo Heap với dữ liệu lớn.

TÀI LIỆU THAM KHẢO

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. MIT Press, 2022.
- [2] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Addison-Wesley, 2011.
- [3] React Team, “React Documentation.” <https://react.dev/> (truy cập 04/08/2026).
- [4] Vite Team, “Vite Guide.” <https://vite.dev/guide/> (truy cập 04/08/2026).
- [5] MDN Web Docs, “HTMLAudioElement.” <https://developer.mozilla.org/docs/Web/API/HTMLAudioElement> (truy cập 04/08/2026).
- [6] Creative Commons, “About CC Licenses.” <https://creativecommons.org/share-your-work/cclicenses/> (truy cập 04/08/2026).
- [7] Openverse, “Openverse API Documentation.” <https://api.openverse.org/> (truy cập 04/08/2026).
- [8] Spotify, “Developer Policy.” <https://developer.spotify.com/policy> (truy cập 04/08/2026).

CHƯƠNG A. CÁC THUẬT TOÁN TRỌNG TÂM VÀ GIẢI THÍCH TỪNG DÒNG

Phụ lục sử dụng mã giả rút gọn, tương đương với phần hiện thực trong backend C11. Tên hàm, thứ tự kiểm tra và bất biến được giữ nguyên; các bước sao chép dữ liệu và tạo phản hồi giao diện không ảnh hưởng đến thuật toán được lược bỏ.

ID	Thuật toán	Mục đích	Độ phức tạp	Kết quả đối chiếu
A1	Bộ so sánh đa tiêu chí	Xác định thứ tự toàn phần cho hàng đợi	$O(1)$	7 phép kiểm tra Heap
A2	Insert + heapifyUp	Thêm yêu cầu và đưa phần tử ưu tiên lên trên	$O(\log n)$	Thêm/tăng điểm
A3	ChangeVote	Đổi bình chọn, tính lại điểm và chọn hướng heapify	$O(\log n)$	Chuyển trạng thái bình chọn
A4	ExtractMax	Lấy gốc Heap làm bài kế tiếp	$O(\log n)$	Lấy gốc Heap
A5	Remove tùy ý	Xóa yêu cầu giữa Heap, giữ đúng Map chỉ số	$O(\log n)$	Xóa/thu hồi
A6	CDLL add/next/prev	Quản lý lịch sử và lập toàn bộ	Add/Next/Prev $O(1)$	4 phép kiểm tra CDLL
A7	Kiểm tra và chặn	Cửa sổ trượt, bài trùng, hạn mức và thu hồi	TB $O(1) + O(r)$	5 phép kiểm tra SpamGuard
A8	Chuyển bài	Phối hợp Heap với CDLL theo chế độ lặp	$O(\log n)$ $O(1)$	hoặc Kiểm tra trạng thái phát

Bảng A.1. Danh mục thuật toán được phân tích chi tiết.

A.1 Bộ so sánh đa tiêu chí

```
1 compare(a, b): // [D1] negative means a ranks first
2   if a.score != b.score: // [D2] score is the primary key
3     return b.score - a.score // [D3] descending score
4   if a.upvotes != b.upvotes: // [D4] use upvotes when scores tie
5     return b.upvotes - a.upvotes // [D5] descending upvotes
6   aTie = a.shuffleOrder ?? INFINITY // [D6] unshuffled items rank later
7   bTie = b.shuffleOrder ?? INFINITY // [D7] same rule for b
8   if aTie != bTie: // [D8] compare only when different
9     return aTie - bTie // [D9] smaller shuffle order first
10  return a.requestedAt - b.requestedAt // [D10] final FIFO tie-break
```

- D1** Khai báo bộ so sánh; giá trị âm nghĩa là a có ưu tiên cao hơn b.
- D2** Điểm là khóa chính; chỉ xét các tiêu chí phụ khi hai điểm bằng nhau.
- D3** Trả chênh lệch theo thứ tự giảm dần, nên điểm lớn đứng trước.
- D4** Nếu bằng điểm, số lượt tăng bình chọn là khóa thứ hai để ưu tiên mức đồng thuận dương.
- D5** Phần tử có nhiều lượt tăng bình chọn hơn tiếp tục đứng trước.
- D6** Yêu cầu chưa được xáo thứ tự nhận giá trị vô cực, nên không chen trước thứ tự do quản trị viên đã gán.
- D7** Áp dụng cùng quy ước cho phần tử b.
- D8** Chỉ xét thứ tự xáo khi hai giá trị thực sự khác nhau.
- D9** Số thứ tự xáo nhỏ hơn có ưu tiên cao hơn.
- D10** Cuối cùng dùng thời điểm tăng dần để bảo đảm FIFO và kết quả xác định.

Bất biến: bộ so sánh tạo thứ tự nhất quán theo bộ (điểm ↓, lượt tăng ↓, *shuffleOrder* ↑, *requestedAt* ↑). Mọi hàm xếp hạng và Heap phải dùng cùng bộ so sánh để giao diện không hiển thị khác gốc Heap.

A.2 Thêm phần tử và *heapifyUp*

```

1 insert(item): // [D1] receive a valid request
2   heap.push(clone(item)) // [D2] isolate external references
3   indexMap[item.id] = heap.length - 1 // [D3] record O(1) lookup position
4   i = heap.length - 1 // [D4] start at the new leaf
5   while i > 0: // [D5] stop at the root
6     p = floor((i - 1) / 2) // [D6] compute parent index
7     if not higherPriority(heap[i], heap[p]): // [D7] invariant already holds
8       break // [D8] terminate early
9     swap(heap[i], heap[p]) // [D9] move item up one level
10    indexMap[heap[i].id] = i // [D10] fix descending item index
11    indexMap[heap[p].id] = p // [D11] fix ascending item index
12    i = p // [D12] continue from parent

```

- D1** Nhận một yêu cầu đã có ID duy nhất và điểm đã được tính.
- D2** Sao chép trước khi lưu để bên gọi không thể sửa ngầm phần tử trong Heap.
- D3** Ghi vị trí mới vào Hash Map chỉ số, cho phép tìm yêu cầu trong $O(1)$ trung bình.
- D4** Bắt đầu tại lá vừa thêm ở cuối mảng.
- D5** Gốc không có cha; vòng lặp dừng khi $i=0$.
- D6** Tính vị trí cha trong biểu diễn mảng của cây nhị phân.
- D7** Nếu con không ưu tiên hơn cha thì bất biến đã đúng trên toàn đường đi.
- D8** Dừng sớm, tránh các phép so sánh không cần thiết.
- D9** Đổi con với cha để đưa phần tử ưu tiên cao lên một mức.
- D10** Sau khi đổi chỗ, cập nhật vị trí phần tử vừa đi xuống.
- D11** Cập nhật vị trí phần tử vừa đi lên; thiếu hai dòng này làm Map chỉ số sai.
- D12** Tiếp tục kiểm tra từ vị trí cha cũ cho đến gốc hoặc điểm ổn định.

Đúng đắn: trước khi thêm, Heap hợp lệ. Chỉ đường từ lá mới đến gốc có thể vi phạm; mỗi lần đổi chỗ sửa một cạnh vi phạm và không phá các cây con. Chiều cao Heap là $\lfloor \log_2 n \rfloor$, nên thời gian $O(\log n)$, bộ nhớ phụ $O(1)$.

A.3 Thay đổi bình chọn và chọn hướng tái cân bằng

```

1  changeVote(requestId, studentId, nextVote): // [D1] target vote is -1, 0, or
    1
2  i = indexMap.get(requestId) // [D2] avoid a linear scan
3  if i is missing: return null // [D3] request already left queue
4  oldVote = heap[i].votes[studentId] ?? 0 // [D4] absent vote equals zero
5  if oldVote == nextVote: return delta 0 // [D5] idempotent transition
6  votes = clone(heap[i].votes) // [D6] preserve input state
7  if nextVote == 0: delete votes[studentId] // [D7] zero removes the key
8  else: votes[studentId] = nextVote // [D8] store upvote/downvote
9  updated = rebuildItem(heap[i], votes) // [D9] rebuild score from snapshot
10 delta = nextVote - oldVote // [D10] supports direct switches
11 heap[i] = updated // [D11] replace at current index
12 if delta > 0: heapifyUp(i) // [D12] score increased: move up
13 else: heapifyDown(i) // [D13] score decreased: move down
14 j = indexMap.get(requestId) // [D14] reread index after heapify
15 return clone(heap[j]), delta // [D15] return command result

```

- D1** Hàm nhận yêu cầu, người bình chọn và trạng thái đích thuộc tập $\{-1, 0, 1\}$.
- D2** Map chỉ số loại bỏ bước quét tuyến tính qua hàng đợi.
- D3** Yêu cầu có thể đã được lấy hoặc xóa; trả `null` giúp API phản hồi an toàn.
- D4** Bình chọn chưa tồn tại được xem là 0.
- D5** Bình chọn cùng chiều lần hai là thao tác lặp an toàn; điểm không đổi.
- D6** Sao chép bản đồ bình chọn để không sửa trạng thái đầu vào.
- D7** Trạng thái 0 được biểu diễn bằng việc xóa khóa, làm gọn ảnh chụp trạng thái.
- D8** Hai trạng thái còn lại được lưu trực tiếp.
- D9** Tính lại số lượt tăng, lượt giảm và điểm từ toàn bộ bình chọn thay vì cộng dồn dễ sai.
- D10** Delta đúng cả chuyển trực tiếp $1 \rightarrow -1$ (-2) và $-1 \rightarrow 1$ ($+2$).
- D11** Thay phần tử tại đúng chỉ số nhưng chưa đổi cấu trúc cây.
- D12** Độ lệch dương chỉ có thể vi phạm với cha, nên tái cân bằng lên trên.
- D13** Độ lệch âm chỉ có thể vi phạm với con, nên tái cân bằng xuống dưới.
- D14** Quá trình tái cân bằng có thể đổi chỉ số; phải đọc lại vị trí từ Map.
- D15** Trả bản sao phần tử ở vị trí mới cùng độ lệch để dịch vụ tạo phản hồi JSON.

Độ phức tạp: tìm chỉ số trong $O(1)$ trung bình, tái tạo bình chọn $O(v)$ với v là số người đã bình chọn yêu cầu, và tái cân bằng $O(\log n)$. Trong mô hình lớp học, v bị chặn bởi số tài khoản; phần thay đổi cấu trúc Heap vẫn là $O(\log n)$.

A.4 ExtractMax

```

1 extractMax(): // [D1] remove highest-priority item
2   if heap.length == 0: return null // [D2] guard an empty Heap
3   max = heap[0] // [D3] root is the next track
4   last = heap.pop() // [D4] keep the array contiguous
5   indexMap.delete(max.id) // [D5] delete stale mapping
6   if heap.length > 0: // [D6] restore if items remain
7     heap[0] = last // [D7] fill the root hole
8     indexMap[last.id] = 0 // [D8] fix index before heapify
9     heapifyDown(0) // [D9] restore downward
10  return clone(max) // [D10] hide internal reference

```

- D1** Lấy và xóa phần tử ưu tiên cao nhất.
- D2** Heap rỗng trả null; không truy cập index 0 ngoài miền.
- D3** Bất biến Heap bảo đảm phần tử gốc chính là bài cần phát.
- D4** Lấy phần tử cuối để giữ mảng liên tục.
- D5** Yêu cầu đã rời Heap nên phải xóa khỏi Map chỉ số.
- D6** Nếu phần tử vừa lấy là phần tử duy nhất thì không cần khôi phục cây.
- D7** Đặt lá cuối vào vị trí trống ở gốc.
- D8** Ghi lại chỉ số 0 trước khi tái cân bằng.
- D9** Gốc mới chỉ có thể vi phạm với con, vì vậy dùng `heapifyDown`.
- D10** Trả bản sao để tầng HTTP không giữ tham chiếu nội bộ.

Độ phức tạp: $O(\log n)$ thời gian và $O(1)$ bộ nhớ phụ. Peek riêng lẻ vẫn $O(1)$; extract phải trả lại bất biến sau khi xóa.

A.5 Xóa một yêu cầu bất kỳ

```

1 remove(requestId): // [D1] admin remove or spam purge
2   i = indexMap.get(requestId) // [D2] average O(1) lookup
3   if i is missing: return null // [D3] repeated delete is safe
4   removed = heap[i] // [D4] retain record for audit
5   last = heap.pop() // [D5] take leaf to fill the hole
6   indexMap.delete(requestId) // [D6] remove stale mapping
7   if i < heap.length: // [D7] skip if last was removed
8     heap[i] = last // [D8] fill the vacant index
9     indexMap[last.id] = i // [D9] synchronize the Map
10    heapifyUp(i) // [D10] test violation with parent
11    j = indexMap.get(last.id) // [D11] reread the new index
12    heapifyDown(j) // [D12] test violation with child
13  return clone(removed) // [D13] return removed request

```

- D1** Xóa theo ID yêu cầu, dùng khi quản trị viên xóa hoặc SpamGuard thu hồi.
- D2** Tra vị trí trung bình $O(1)$ bằng Map.
- D3** Bảo đảm thao tác lặp lại vẫn an toàn khi yêu cầu đã biến mất.
- D4** Giữ phần tử bị xóa để ghi nhật ký.
- D5** Lấy phần tử cuối nhằm lấp lỗ mà không dịch mảng $O(n)$.
- D6** Xóa ánh xạ của phần tử không còn tồn tại.
- D7** Nếu xóa đúng phần tử cuối, thao tác lấy cuối đã hoàn tất việc xóa.
- D8** Đưa phần tử cuối vào vị trí trống.
- D9** Đồng bộ Map ngay sau khi thay vị trí.
- D10** Ưu tiên mới có thể lớn hơn cha, nên thử `heapifyUp` trước.
- D11** Đọc lại chỉ số vì phần tử có thể đã đi lên.
- D12** Từ vị trí mới, thử `heapifyDown` để bao phủ trường hợp nhỏ hơn con.
- D13** Trả bản sao yêu cầu đã xóa.

Lý do chạy hai hướng: phần tử cuối thay thế một vị trí bất kỳ, nên chưa biết nó vi phạm với cha hay với con. Sau `heapifyUp`, đọc lại chỉ số rồi dùng `heapifyDown` bảo đảm đúng cả hai trường hợp mà vẫn $O(\log n)$.

A.6 Danh sách liên kết đôi vòng

```

1  addLast(value): // [D1] append track to history
2      node = new Node(value) // [D2] allocate a new node
3      if head == null: // [D3] list is empty
4          node.prev = node; node.next = node // [D4] valid one-node ring
5          head = node; current = node // [D5] initialize head/current
6      else: // [D6] list is non-empty
7          tail = head.prev // [D7] get tail in O(1)
8          tail.next = node; node.prev = tail // [D8] link to old tail
9          node.next = head; head.prev = node // [D9] close ring with head
10     length = length + 1 // [D10] increment after linking
11
12 next(): if current != null: current = current.next // [D12] advance and wrap
13 prev(): if current != null: current = current.prev // [D13] rewind and wrap

```

- D1** Nói một bài đã phát vào cuối lịch sử.
- D2** Cấp phát nút mới; chỉ số của nút bằng độ dài trước khi tăng.
- D3** Tách trường hợp rỗng vì chưa có nút cuối.
- D4** Một nút tự trở cả hai chiều, tạo vòng hợp lệ có kích thước một.
- D5** Nút đầu và nút hiện hành cùng trở đến phần tử đầu tiên.
- D6** Nhánh danh sách không rỗng.
- D7** Trong CDLL, nút cuối luôn là `head.prev`, truy cập trong $O(1)$.
- D8** Nối nút cuối cũ sang nút mới và nút mới lùi về nút cuối cũ.
- D9** Khép vòng: nút mới tiến về nút đầu, còn nút đầu lùi về nút mới.
- D10** Tăng kích thước sau khi bốn liên kết đã nhất quán.
- D12** Next chỉ đổi một con trỏ; tại nút cuối, `next=head` tạo chế độ lặp toàn bộ tự nhiên.
- D13** Previous hoạt động đối xứng; tại nút đầu, `prev=tail`.

Bất biến: với mọi nút x , $x.next.prev=x$ và $x.prev.next=x$; `head.prev` là `tail` và `tail.next` là `head`. `AddLast`, `Next` và `Prev` đều $O(1)$.

A.7 Cửa sổ trượt, chặn và thu hồi

```
1 checkBeforeRequest(student, song, now): // [D1] run before touching Heap
2   state = getOrCreate(student) // [D2] get/create StudentID state
3   state.recent = filter(t >= now - WINDOW) // [D3] prune outside 10 minutes
4   if state.blockedUntil > now: // [D4] prioritize active block
5     return BLOCKED with no new purge // [D5] avoid a second purge
6   if exists(state.recent, same song): // [D6] find canonical duplicate
7     return blockStudent(DUPLICATE, now) // [D7] duplicate: block 30 minutes
8   if state.recent.length >= MAX_REQUESTS: // [D8] enforce request limit
9     return blockStudent(LIMIT, now) // [D9] fourth request is blocked
10  return ALLOWED // [D10] all guards passed
11
12 blockStudent(reason, now): // [D12] atomic block update
13   purgeIds = clone(state.activeRequestIds) // [D13] snapshot IDs to revoke
14   state.blockedUntil = now + BLOCK_DURATION // [D14] compute block deadline
15   state.reason = reason; state.blockCount += 1 // [D15] store reason and count
16   state.recent = []; state.activeRequestIds = [] // [D16] reset to avoid
    repurge
17  return BLOCKED(reason, blockedUntil, purgeIds) // [D17] return service
    payload
```

- D1** Kiểm tra trước khi dịch vụ C thay đổi Heap.
- D2** Hash Map trả trạng thái của StudentID hoặc tạo trạng thái rỗng.
- D3** Cửa sổ trượt loại mốc cũ; chỉ yêu cầu trong 10 phút còn hiệu lực.
- D4** Trạng thái chặn đang hiệu lực được ưu tiên trước kiểm tra trùng và hạn mức.
- D5** Không thu hồi lần hai vì lần chặn đầu đã trả và xóa danh sách đang hoạt động.
- D6** Tìm khóa bài hát chuẩn hóa trùng nhau trong cửa sổ hiện hành.
- D7** Yêu cầu trùng kích hoạt thời gian chặn 30 phút.
- D8** Nếu không trùng, kiểm tra ngưỡng số yêu cầu.
- D9** Yêu cầu tiếp theo sau ba yêu cầu hợp lệ kích hoạt trạng thái chặn.
- D10** Chỉ khi qua toàn bộ điều kiện mới trả kết quả cho phép.
- D12** Hàm chặn thực hiện cập nhật trạng thái nguyên tử.
- D13** Sao chép ID trước khi xóa để backend biết bài nào cần thu hồi khỏi Heap.
- D14** Tính mốc hết chặn tuyệt đối từ now.

- D15 Lưu lý do và tăng bộ đếm sự cố phục vụ phân tích.
- D16 Đặt lại cửa sổ và danh sách sở hữu đang hoạt động, tránh thu hồi lặp.
- D17 Trả đủ lý do, thời điểm và ID; backend gọi hàm xóa từng yêu cầu khỏi Heap.

Độ phức tạp: tra Hash Map trung bình $O(1)$; loại móc cũ và dò yêu cầu trùng tồn $O(r)$ với r là số yêu cầu trong cửa sổ. Thu hồi k yêu cầu khỏi Heap tồn $O(k \log n)$; xóa bình chọn của sinh viên cần dựng lại Heap trong $O(n)$ khi có vi phạm.

A.8 Chọn bài kế tiếp và phối hợp hai CTDL

```

1 advance(state, now, manual): // [D1] Next or audio ended
2   if repeat == ONE and not manual: // [D2] Repeat One is automatic only
3     return state with progress = 0 // [D3] replay from the start
4   heap = QueueMaxHeap(state.queue) // [D4] restore a pure Heap
5   item = heap.extractMax() // [D5] queue precedes history
6   if item exists: // [D6] a queued track exists
7     list = CDLL.fromArray(history, currentIndex) // [D7] restore current node
8     added = list.addLast(item.song) // [D8] append to history
9     return queue=heap, history=list, current=added // [D9] commit atomically
10  if repeat == OFF and not manual: // [D10] empty queue, no repeat
11    return stopped // stop automatic transition
12  if history is not empty: // [D11] Repeat All has data
13    moved = CDLL(history, currentIndex).next() // [D12] wrap tail to head
14    return current=moved, progress=0 // [D13] move and reset progress
15  return stopped with EMPTY warning // [D14] queue and history empty

```

- D1 Điều phối khi quản trị viên nhấn Next hoặc âm thanh phát hết.
- D2 Lặp một bài chỉ áp dụng cho chuyên tự động; Next thủ công vẫn đổi bài.
- D3 Đưa vị trí phát về đầu bài hiện tại.
- D4 Khôi phục Heap từ ảnh chụp trạng thái để thao tác thuận và có thể kiểm thử.
- D5 Lấy phần tử gốc; nếu có bài chờ, Heap luôn được ưu tiên hơn lịch sử vòng.
- D6 Tách nhánh có phần tử để cập nhật đồng thời hàng đợi và lịch sử phát.
- D7 Khôi phục CDLL với đúng chỉ số hiện hành.
- D8 Nối bài vừa lấy vào cuối lịch sử; chỉ số trả về là vị trí phát mới.
- D9 Ghi nhận đồng thời hàng đợi sau khi lấy, lịch sử sau khi thêm và bài hiện hành.
- D10 Khi hàng đợi rỗng và tắt chế độ lặp, kết thúc phát tự động.
- D11 Lặp toàn bộ chỉ có ý nghĩa nếu lịch sử không rỗng.
- D12 Liên kết next của CDLL tự nối nút cuối về nút đầu trong $O(1)$.
- D13 Reset tiến độ khi chuyển bài.
- D14 Hàng đợi và lịch sử đều rỗng: dừng an toàn và trả phản hồi rõ ràng.
- D15 Backend hoàn tất đồng thời hàng đợi, lịch sử và chỉ số hiện hành trước khi tạo snapshot JSON; giao diện không quan sát trạng thái trung gian giữa Heap và lịch sử.

CHƯƠNG B. MA TRẬN TRUY VẾT YÊU CẦU

ID	Yêu cầu	Module chính	Cơ chế/CTDL	Kết quả kiểm tra
F1	Một hàng đợi công cộng	backend-c/src/ backend.c	Snapshot tập trung trong RAM	API state + bản dựng
F2	Tìm và yêu cầu bài	catalog.tsx, backend C	Kiểm tra danh mục, khóa bài chuẩn hóa, thêm vào Heap	Kiểm thử danh mục + API
F3	Phát, Next, Prev, Repeat	useAudioPlayer. ts, backend C	CDLL, con trỏ current, liên kết vòng	4 phép kiểm tra CDLL
F4	Tăng/giảm bình chọn	backend-c/src/ heap.c	Độ chênh bình chọn, Map chỉ số, heapify hai hướng	7 phép kiểm tra Heap
F5	Chống gửi quá mức	backend-c/src/ spam_guard.c	Hash Map, cửa sổ trượt, ID yêu cầu đang hoạt động	5 phép kiểm tra SpamGuard
F6	Quản trị hàng đợi	App.tsx, backend C	Xóa, trộn, Next và reset qua API	Luồng trình duyệt + bản dựng
NF1	Giao diện thích ứng/PWA	App.css, manifest, SW	Container query, breakpoint, service worker	Đo tại 1200/390 px
NF2	Khả năng tái lập	tests/, backend-c/tests/	Dữ liệu cố định, assert, lệnh npm	22/22 + 56 assertion C đạt

Bảng B.1. Truy vết từ yêu cầu đến mã nguồn và kết quả kiểm tra.

Độ phủ truy vết

Ma trận bao phủ toàn bộ 6 yêu cầu chức năng và 2 yêu cầu phi chức năng được công bố trong phạm vi bản nộp. Mỗi hàng đồng thời chỉ ra nơi hiện thực, cơ chế CTDL và kết quả xác minh; vì vậy một yêu cầu chỉ được xem là “đã truy vết” khi đủ cả ba liên kết.

8/8

yêu cầu có module

8/8

yêu cầu có cơ chế

8/8

yêu cầu có kết quả kiểm tra

$$T_{trace} = \frac{N_{\text{đủ module, cơ chế và kiểm tra}}}{N_{\text{yêu cầu đã công bố}}} = \frac{8}{8} = 100\%.$$

Quy ước quản trị thay đổi

- ID yêu cầu được giữ ổn định qua các lần sửa; thay đổi cách diễn đạt không tạo ID mới nếu hành vi nghiệm thu không đổi.
- Một ảnh chụp chỉ chứng minh trạng thái hiển thị, không thay thế kiểm thử đơn vị cho bất biến CTDL hoặc kiểm tra dữ liệu tại backend.
- Khi module chính thay đổi, chủ sở hữu yêu cầu phải cập nhật đồng thời phép kiểm tra, kết quả và hàng tương ứng trong Bảng B.1.
- Yêu cầu chưa có phép kiểm tra tự động phải ghi rõ tình huống trình duyệt, kích thước màn hình hoặc nhật ký dùng để xác minh thủ công.

Kết luận. Ma trận đạt độ phủ truy vết 100% trong phạm vi yêu cầu đã công bố. Chi số này phản ánh mức đầy đủ của liên kết giữa tài liệu, mã nguồn và kiểm thử; không phải tỷ lệ bao phủ theo dòng lệnh và cũng không mở rộng kết luận sang các hành vi ngoài phạm vi.

CHƯƠNG C. ĐẶC TẢ BẤT BIẾN VÀ ĐIỀU KIỆN KIỂM CHỨNG

Phụ lục này chuyển các yêu cầu đúng đắn thành mệnh đề có thể kiểm tra sau mỗi phép biến đổi. Các điều kiện kiểm chứng không dựa vào thứ tự đang hiển thị trên giao diện mà đọc trực tiếp trạng thái của lõi CTDL, nhờ đó tách lõi thuật toán khỏi lõi trình bày.

C.1 Max-Heap

Với mọi $i \in \{1, \dots, n - 1\}$, quan hệ Heap phải thỏa

$$\text{higherPriority}(H[\text{parent}(i)], H[i]) = \text{true}.$$

Đồng thời, Map chỉ số phải là một song ánh trên các phần tử đang có trong Heap:

$$\text{indexMap}[H[i].\text{requestId}] = i \quad \wedge \quad 0 \leq \text{indexMap}[id] < n.$$

Sau thao tác đổi chỗ, xóa hoặc lấy phần tử đầu, hai điều kiện được kiểm tra lại. Các trường hợp bằng điểm được phân giải lần lượt bằng lượt tăng bình chọn, `shuffleOrder` và `requestedAt`; bộ so sánh vì thế tạo một thứ tự toàn phần xác định, không phụ thuộc vào thứ tự hiển thị.

C.2 Danh sách liên kết đôi vòng

Danh sách rỗng có `head = current = null` và `length = 0`. Với danh sách không rỗng, mọi nút x phải thỏa

$$x.\text{next}.\text{prev} = x \quad \wedge \quad x.\text{prev}.\text{next} = x, \quad \text{head}.\text{prev} = \text{tail}, \quad \text{tail}.\text{next} = \text{head}.$$

Duyệt đúng `length` bước từ nút đầu phải trở lại nút đầu mà không gặp `null` hay thăm một nút hai lần trước khi khép vòng. Điều kiện này được dùng cho trường hợp danh sách rỗng, một nút, nhiều nút và khép vòng hai chiều.

C.3 SpamGuard

Mỗi StudentID đã chuẩn hóa ánh xạ đến đúng một trạng thái. Các mốc thời gian sau khi loại dữ liệu cũ phải nằm trong cửa sổ $[now - W, now]$ với $W = 10$ phút; `activeRequestIds`

chỉ chứa yêu cầu còn ở Heap. Khi chặn, kết quả phải trả toàn bộ ID cần thu hồi và `blockedUntil = now + 30 phút`; yêu cầu mới không được thay đổi Heap. Sau khi thu hồi, tập ID hoạt động phải rỗng để cùng một vi phạm không bị xử lý hai lần.

C.4 Tính nguyên tử của trạng thái backend

Một lần chuyển bài cập nhật đồng thời hàng đợi sau `extract`, lịch sử sau `addLast` và `con` trở hiện hành trong tiến trình C. Backend chỉ tạo snapshot sau khi toàn bộ thao tác hoàn tất; giao diện không quan sát thời điểm trung gian khi bài đã rời Heap nhưng chưa được thêm vào lịch sử. Tương tự, thao tác chặn phải cập nhật SpamGuard và xóa toàn bộ `purgeIds` khỏi Heap trước khi trả phản hồi.

Đối tượng	Điều kiện sau thao tác	Trường hợp kiểm tra	Tín hiệu lỗi
Max-Heap	Node cha không kém ưu tiên hơn node con; Map hai chiều nhất quán.	Thêm, bình chọn hai hướng, lấy gốc và xóa giữa.	Sai gốc hoặc chỉ số không còn đúng.
CDLL	Liên kết nghịch đảo đúng; đủ <code>length</code> bước khép vòng.	Danh sách rỗng, một node và Next/Prev qua biên.	Giá trị null ngoài ý muốn hoặc mất node.
SpamGuard	Cửa sổ đã loại dữ liệu cũ; thao tác chặn trả đúng tập ID cần thu hồi.	Yêu cầu thứ tư, bài trùng và hết hạn chặn.	Bài của tài khoản bị chặn còn trong Heap.
Backend C	Hàng đợi, lịch sử và vị trí hiện tại thuộc cùng snapshot.	Next thủ công, audio kết thúc, chặn và thu hồi.	Giao diện quan sát trạng thái chưa hoàn chỉnh.

Bảng C.1. Điều kiện kiểm chứng và dấu hiệu vi phạm bất biến.

CHƯƠNG D. HƯỚNG DẪN TÁI LẬP KẾT QUẢ

D.1 Môi trường tối thiểu

Mã nguồn chính thức được công bố tại github.com/PhamHungTien/HeapBeat dưới giấy phép MIT. Người đọc nên dùng đúng nhánh `main` đã công bố để đối chiếu kết quả trong báo cáo.

Thành phần	Yêu cầu	Vai trò xác minh
Node.js/npm	Node.js 20 trở lên; cài thư viện bằng <code>npm ci</code> .	Cố định phiên bản thư viện, chạy Vitest, TypeScript và Vite.
Trình biên dịch C	Clang hoặc GCC hỗ trợ C11.	Biên dịch với cảnh báo là lỗi và chạy bản đối chiếu độc lập.
Trình duyệt	Chromium/WebKit hiện đại, công cụ mô phỏng kích thước màn hình.	Kiểm tra luồng tích hợp, hiện tượng tràn DOM và PWA.
LaTeX/Poppler	XeLaTeX, <code>latexmk</code> , <code>pdfinfo</code> và <code>pdftoppm</code> .	Biên dịch và kiểm tra PDF A4 không cắt chữ/bảng/ảnh.

Bảng D.1. Môi trường tối thiểu để tái lập các nhóm kết quả.

D.2 Quy trình và đầu ra kỳ vọng

- Cài Node.js 20 trở lên và một trình biên dịch C11 (Clang hoặc GCC).
- Tại thư mục gốc chạy `npm ci`; không sửa dữ liệu kiểm thử hoặc `localStorage` giữa các lần chạy.
- Chạy `npm run test`; kết quả kỳ vọng: 1 tệp kiểm thử, 22/22 phép kiểm tra đạt.
- Chạy `npm run build`; kết quả kỳ vọng: TypeScript và Vite hoàn tất, không lỗi.
- Chạy `npm run test:backend:c`; kết quả kỳ vọng: `[PASS] 56 assertions - HeapBeat backend C`.
- Chạy `npm run dev`; đăng nhập `admin/admin@123`, quan sát gốc Heap trước và sau bình chọn/lấy bài. Kiểm tra ở 1200x820 và 390x844; thân trang và hàng đợi không có cuộn ngang.
- Chạy `npm run report`; dùng `pdfinfo` kiểm tra A4 và kết xuất toàn bộ trang bằng Poppler để xác nhận không có chữ bị cắt, bảng tràn hoặc ảnh mờ.

D.3 Danh sách xác nhận độc lập

ID	Điểm cần xác nhận	Kết quả kỳ vọng
R1	Thư viện được cài từ lockfile, dữ liệu kiểm thử không bị thay đổi.	npm ci hoàn tất.
R2	Toàn bộ kiểm thử CTDL và danh mục đạt.	1 tệp; 22/22 phép kiểm tra.
R3	Bản C không có cảnh báo và mọi lệnh assert đúng.	Chuỗi all assertions passed.
R4	Bản dựng web không có lỗi kiểu hoặc đóng gói.	Vite kết thúc với mã thoát 0.
R5	Gốc Heap đổi đúng sau bình chọn/lấy bài; bài trùng bị chặn và thu hồi.	Quan sát giao diện + nhật ký.
R6	Thân trang và hàng đợi không cuộn ngang ở 1200x820, 390x844.	clientWidth = scrollWidth.
R7	PDF là A4, đủ trang, không tràn hoặc lỗi ký tự.	pdfinfo + ảnh kết xuất.

Điều kiện diễn giải kết quả. Bộ kiểm thử xác nhận hành vi trên dữ liệu và các trường hợp biên đã đặc tả; không phải là chứng minh hình thức cho mọi đầu vào, phép đo tải quy mô lớn hoặc chứng nhận về bảo mật và bản quyền. Kết luận của báo cáo chỉ có giá trị trong phạm vi bản nộp và mã nguồn được công bố.

CHƯƠNG E. KẾ HOẠCH THỰC HIỆN VÀ PHÂN CÔNG NHIỆM VỤ

Kế hoạch được chuẩn hóa theo chín tuần, từ ngày 08/06/2026 đến buổi thuyết trình ngày 08/08/2026. Các giai đoạn có thể chạy song song nhưng mỗi đầu ra đều có một người sở hữu và một người kiểm tra chéo trước khi tích hợp.

E.1 Ma trận phân công cân bằng

Khối lượng được chuẩn hóa thành **12 điểm/người**: 2 điểm phân tích–thiết kế, 4 điểm hiện thực, 2 điểm kiểm thử, 2 điểm tài liệu và minh họa, 2 điểm kiểm tra chéo. Tổng kế hoạch là 96 điểm; mỗi thành viên đảm nhiệm $12/96 = 12,5\%$ và sở hữu một đầu ra có thể nghiệm thu độc lập.

8

thành viên

12

điểm/người

12,5%

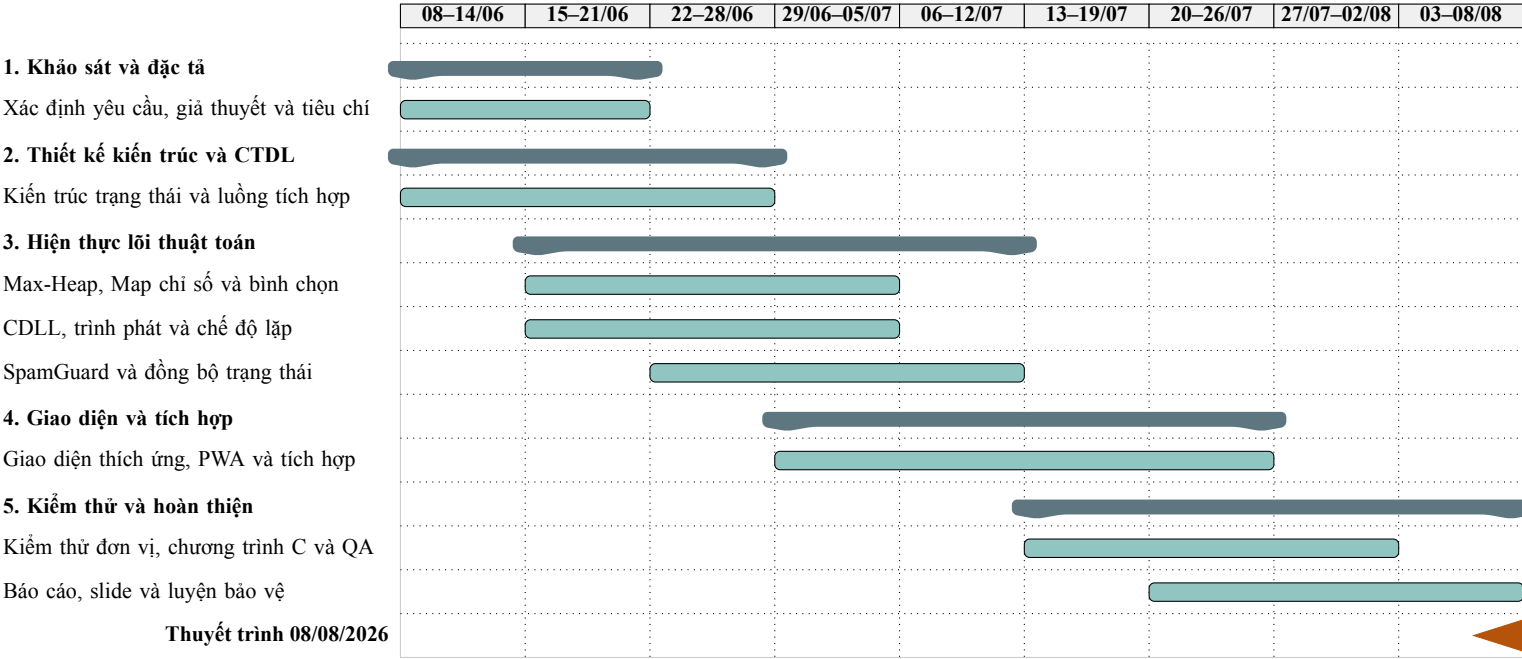
tỷ trọng/người

Thành viên	Gói công việc sở hữu	Kiểm tra chéo	Đầu ra nghiệm thu	Điểm
Phạm Hùng Tiến	Kiến trúc trạng thái, tích hợp module và đóng gói	PHP proxy và cấu hình backend C	Ứng dụng tích hợp, lệnh đóng gói và danh sách kiểm tra	12
Trần Thái Nguyên	CDLL, trình phát và Next/Prev/Repeat	Giao diện điện thoại và vòng đời âm thanh	CDLL, trình phát và 4 phép kiểm tra vòng	12
Quách Gia Thịnh	Max-Heap, bộ so sánh, bình chọn và xóa	Chương trình C và bất biến Heap	QueueMaxHeap và 7 phép kiểm tra Heap	12
Đặng Gia Toàn	SpamGuard, cửa sổ trượt, chặn và thu hồi	Dịch vụ request của backend C	SpamGuard và 5 phép kiểm tra chống gửi quá mức	12
Phạm Cao Bằng	Giao diện thích ứng, hàng đợi và khả năng truy cập	Danh mục và hộp thoại	CSS breakpoint/container query và số đo màn hình	12
Lê Quang Bảo Trung	Cổng API PHP-C và snapshot JSON	Phục hồi trạng thái và xung đột cập nhật	PHP proxy và kịch bản chạy backend C trên LAN	12
Văn Phú Duy	Kế hoạch kiểm thử, chương trình C, QA và số liệu	Max-Heap và SpamGuard	22 kiểm thử đơn vị, lệnh assert C và biên bản QA	12
Hồ Sĩ Phú	Danh mục, hồ sơ giấy phép và báo cáo khoa học	Ma trận truy vết và hướng dẫn chạy	Danh mục 18 bài, hồ sơ giấy phép và PDF	12
Tổng				96

Bảng E.1. Ma trận phân công cân bằng theo điểm công việc và kiểm tra chéo.

E.2 Biểu đồ Gantt

Kế hoạch thực hiện dự án HeapBeat
Thời gian: 08/06/2026 – 08/08/2026 | Ngày thuyết trình: 08/08/2026



E.3 Mốc kiểm soát và tiêu chí hoàn thành

Mốc	Thời điểm	Đầu ra bắt buộc	Điều kiện đóng mốc
M1 – Đặc tả	21/06/2026	Yêu cầu, giả thuyết, tiêu chí nghiệm thu và dữ liệu minh họa.	Nhóm duyệt chéo, không còn yêu cầu mơ hồ.
M2 – Lỗi CTDL	05/07/2026	Max-Heap, CDLL và SpamGuard hoạt động độc lập.	Bất biến và trường hợp biên được kiểm tra.
M3 – Tích hợp	26/07/2026	Luồng yêu cầu, bình chọn, phát bài và giao diện thích ứng.	Bản dựng web hoàn tất trên desktop/mobile.
M4 – Đóng băng	02/08/2026	22 kiểm thử, bản C, hồ sơ bản quyền và số liệu QA.	Không còn lỗi nghiêm trọng hoặc lỗi tràn bộ cục.
M5 – Bảo vệ	08/08/2026	Báo cáo, slide, kịch bản và bản demo có thể tái lập.	Tổng duyệt đúng thời lượng; hồ sơ bàn giao được kiểm tra lần cuối.

Bảng E.2. Các mốc kiểm soát dùng để quyết định chuyển giai đoạn.

E.4 Quy tắc bàn giao và kiểm tra chéo

1. Chủ sở hữu bàn giao mã nguồn, phép kiểm tra và mô tả đầu ra trong cùng một lần duyệt.
2. Người kiểm tra chéo xác nhận bất biến, tình hướng biên và khả năng tái lập trước khi tích hợp.
3. Thay đổi sau mốc đóng băng phải ghi rõ ảnh hưởng đến mã nguồn, kiểm thử, báo cáo và slide.
4. Nhóm trưởng chỉ đóng gói khi tám đầu ra đều đạt và không còn xung đột giữa tài liệu với sản phẩm.